

BiLSTM with Attention

Class Definition

```
1 class BiLSTMAttention(nn.Module):
```

Creates BiLSTM model with attention mechanism.

Constructor

```
1     def __init__(self, vocab_size, embedding_dim=256, hidden_dim=128,
2                   num_layers=2, output_dim=3, dropout=0.5):
```

Initialization with parameters:

- vocab_size: Number of unique words
- embedding_dim=256: Word vector size
- hidden_dim=128: LSTM hidden state size per direction
- num_layers=2: Stacked LSTM layers
- output_dim=3: Sentiment classes
- dropout=0.5: Dropout rate

```
1     super().__init__()
```

Initializes parent class.

```
1     self.embedding = nn.Embedding(vocab_size, embedding_dim,
                                   padding_idx=0)
```

Creates embedding layer. Shape: (vocab_size, 256).

```
1     self.bilstm = nn.LSTM(
2         input_size=embedding_dim,
3         hidden_size=hidden_dim,
4         num_layers=num_layers,
5         batch_first=True,
6         dropout=dropout if num_layers > 1 else 0,
7         bidirectional=True
8     )
```

Creates bidirectional LSTM:

- Input: 256 dimensions

- Hidden: 128 per direction (256 total)
- 2 layers with dropout between them
- Processes sequences both forward and backward

```

1         self.attention = nn.Sequential(
2             nn.Linear(hidden_dim * 2, hidden_dim),
3             nn.Tanh(),
4             nn.Linear(hidden_dim, 1)
5         )

```

Creates attention mechanism with 3 sequential layers:

- Layer 1: Linear ($256 \rightarrow 128$) - reduces dimensionality
- Layer 2: Tanh activation - non-linearity
- Layer 3: Linear ($128 \rightarrow 1$) - produces attention score per time step

```

1         self.dropout = nn.Dropout(dropout)

```

Creates dropout layer.

```

1         self.fc = nn.Linear(hidden_dim * 2, output_dim)

```

Creates output layer ($256 \rightarrow 3$).

Forward Method

```

1     def forward(self, input_ids, attention_mask):

```

Defines forward pass with required inputs.

```

1         # Embedding
2         embedded = self.dropout(self.embedding(input_ids))

```

Converts indices to embeddings with dropout. Shape: (batch, sequence, 256).

```

1         # BiLSTM
2         bilstm_out, _ = self.bilstm(embedded)

```

Processes through BiLSTM. Output shape: (batch, sequence, 256). Hidden states ignored.

```

1         # Attention additive
2         attn_scores = self.attention(bilstm_out).squeeze(-1)

```

Computes attention scores for each time step. Shape: (batch, sequence, 1) $\rightarrow$ (batch, sequence).

```

1         attn_scores = attn_scores.masked_fill(attention_mask == 0, -1e9)

```

Masks padding positions by setting their scores to very negative value (-1e9) so they get zero weight after softmax.

```

1         attn_weights = F.softmax(attn_scores, dim=1).unsqueeze(1)

```

Converts scores to probabilities using softmax, then adds dimension. Shape: (batch, sequence) → (batch, 1, sequence).

```
1      # Context vector pondere
2      context_vector = torch.bmm(attn_weights, bilstm_out).squeeze(1)
```

Computes weighted sum of LSTM outputs using attention weights via batch matrix multiplication. Shape: (batch, 1, 256) → (batch, 256).

```
1      # Classification
2      return self.fc(self.dropout(context_vector))
```

Applies dropout and linear layer to produce 3 logits.

Device Selection

```
1 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

Selects GPU if available, otherwise CPU.

```
1 print(f"      Device: {device}")
```

Displays selected device.

Model Instantiation

```
1 model = BiLSTMAttention(
2     vocab_size=len(vocab),
3     embedding_dim=256,
4     hidden_dim=128,
5     num_layers=2,
6     output_dim=3,
7     dropout=0.5
8 ).to(device)
```

Creates model instance and moves it to selected device.

```
1 print(f" Modele BiLSTM + Attention initialise :")
2 print(f"      - Type          : Bidirectionnel (forward + backward)")
3 print(f"      - Embedding dim    : 256")
4 print(f"      - Hidden dim       : 128 ( 2 = 256 avec bidirection)")
5 print(f"      - Couches          : 2")
6 print(f"      - Parametres       : {sum(p.numel() for p in model.parameters())
7      },}")
```

Prints model configuration and parameter count.

Tools Used

PyTorch Core (torch)

- torch.device: Specifies computation device (CPU/GPU)
- torch.bmm: Batch matrix multiplication for attention

Neural Network Module (nn)

- `nn.Module`: Base class
- `nn.Embedding`: Word embeddings
- `nn.LSTM`: Bidirectional LSTM
- `nn.Sequential`: Chains layers (used for attention)
- `nn.Linear`: Fully connected layers
- `nn.Tanh`: Activation function
- `nn.Dropout`: Regularization

Functional Interface (F)

- `F.softmax`: Converts scores to probabilities

Key Differences from Basic BiLSTM

- **Attention mechanism**: Learns which time steps are important instead of just using final hidden state
- **Context vector**: Weighted combination of all LSTM outputs
- **Masking**: Ensures padding doesn't affect attention weights